
Sebenta

Aula 05 — Bluetooth Low Energy: Programação

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026–2027

Catarina Silva

ESTGA, Universidade de Aveiro

Texto de apoio ao estudo autónomo

Índice

Como Usar Esta Sebenta	2
1 Comentário aos Diapositivos	2
2. Agenda (1/2)	2
3. Agenda (2/2)	3
4. Arquitetura da Cadeia Completa	4
5. Arquitetura do Exercício (1/2)	5
6. Arquitetura do Exercício (2/2)	6
7. Bibliotecas e Wrappers para BLE (1/2)	7
8. Bibliotecas e Wrappers para BLE (2/2)	8
9. bluepy: Modelo de Classes (1/2)	9
10. bluepy: Modelo de Classes (2/2)	10
11. Implementação de um Varredor (1/2)	11
12. Implementação de um Varredor (2/2)	12
13. Papéis e Sentido das Trocas	13
14. Notificações: Conceito (1/2)	14
15. Notificações: Conceito (2/2)	15
16. Operações GATT e Custo Energético	16
17. Notificações: Implementação em bluepy (1/2)	17
18. Notificações: Implementação em bluepy (2/2)	18
19. O Shield BLE HM-11 (1/2)	19
20. O Shield BLE HM-11 (2/2)	20
21. Estrutura do Sketch Arduino	21
22. Estrutura do Programa Arduino (1/2)	22
23. Estrutura do Programa Arduino (2/2)	23
24. Verificação do Lado do Gateway (1/2)	24
25. Verificação do Lado do Gateway (2/2)	25
26. Método de Trabalho Recomendado (1/2)	26
27. Método de Trabalho Recomendado (2/2)	27
28. Síntese da Aula (1/2)	28
29. Síntese da Aula (2/2)	29
30. Fontes	30
Verificação Final	31
Referências	31

Como Usar Esta Sebenta

Este texto acompanha, diapositivo a diapositivo, a exposição da Aula 05. Cada página reproduz um diapositivo e indica, por baixo, o que dele se deve retirar.

A leitura não substitui a aula nem os diapositivos. Destina-se ao estudo posterior, quando o estudante já ouviu a exposição e pretende consolidar, verificar se compreendeu o essencial ou preparar a avaliação. Os comentários explicitam ligações entre diapositivos que na exposição oral ficam implícitas, e assinalam os pontos em que a matéria costuma ser mal compreendida.

Sempre que um comentário formula uma pergunta, essa pergunta é o critério de verificação: se não conseguir respondê-la, convém voltar ao diapositivo antes de avançar.

1. Comentário aos Diapositivos

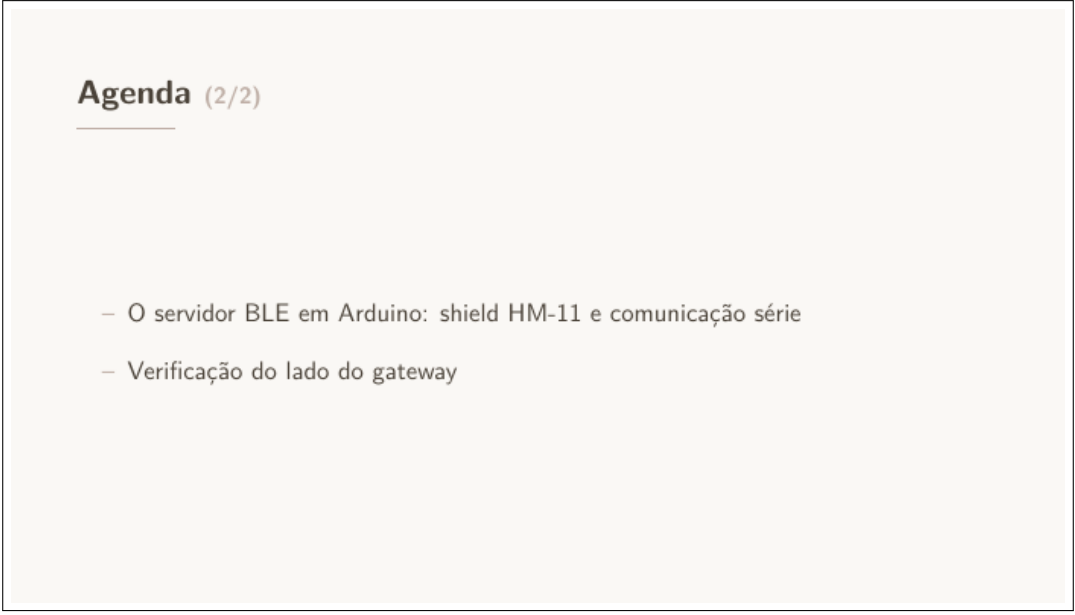
Diapositivo 2. Agenda (1/2)

Agenda (1/2)

- Desenvolvimento de aplicações BLE: arquitetura do exercício
- Bibliotecas e wrappers disponíveis: BlueZ, bluepy, PyGATT e SimpleBLE
- A biblioteca bluepy: modelo de classes
- Implementação de um varredor (*scanner*)
- Trabalho com notificações

O percurso da aula. A aula tem estrutura descendente. Parte da arquitetura do sistema a construir, desce às bibliotecas disponíveis, desce depois ao modelo de classes de uma delas e termina no código concreto. Esta ordem serve um propósito: evita que o estudante comece a escrever instruções antes de saber que peças está a articular.

Diapositivo 3. Agenda (2/2)

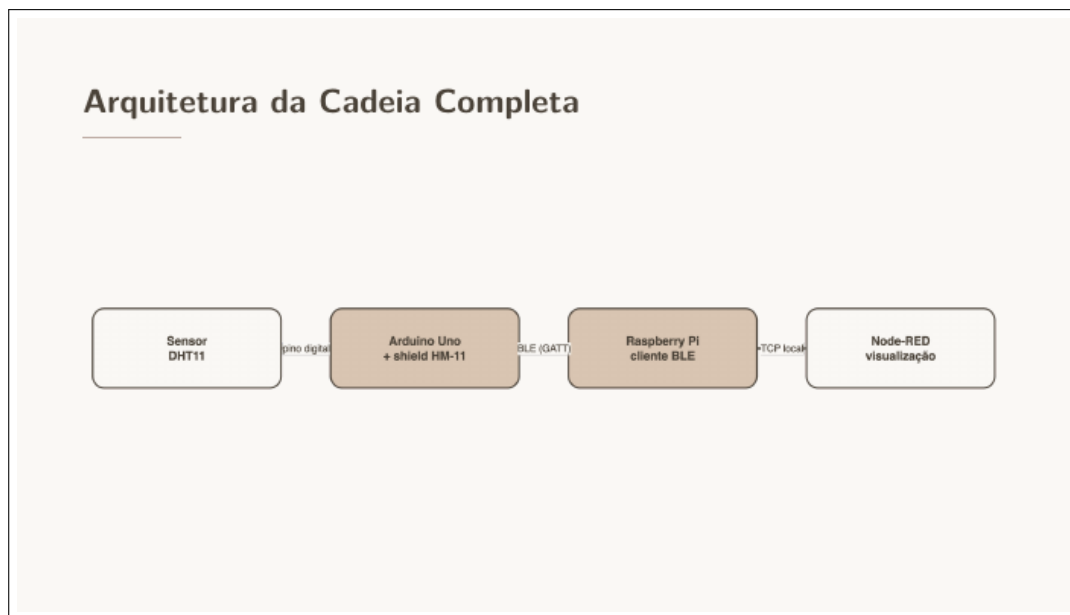


Agenda (2/2)

- O servidor BLE em Arduino: shield HM-11 e comunicação série
- Verificação do lado do gateway

A segunda metade. Os dois últimos temas mudam de lado. Passam do cliente, que se programa, para o servidor, que se monta, e terminam na verificação a partir do gateway.

Diapositivo 4. Arquitetura da Cadeia Completa



O sistema completo, de uma só vez. O diagrama percorre os quatro elementos da cadeia e indica o protocolo de cada troço. Vale a pena fixá-lo antes de avançar, porque todos os diapositivos seguintes tratam de um único troço de cada vez. Repare-se que só o primeiro troço é BLE. Do gateway para a frente, a comunicação já é sobre IP.

Diapositivo 5. Arquitetura do Exercício (1/2)

Arquitetura do Exercício (1/2)

- O objetivo desta aula é implementar, na prática, os conceitos de GAP e GATT vistos na aula anterior.
- Do lado do sensor, um Arduino equipado com um shield BLE assume o papel de periférico, ou seja, de servidor GATT.
- Do lado do gateway, um computador com Linux (nas aulas seguintes, um Raspberry Pi) assume o papel de central, ou seja, de cliente GATT.

Quem faz o quê. O Arduino com shield BLE assume o papel de periférico, ou seja, de servidor GATT. O computador com Linux assume o de central, ou seja, de cliente GATT. Esta correspondência entre papéis GAP e papéis GATT não é obrigatória pela norma, mas é a configuração habitual e é a que se usa em toda a unidade curricular.

Diapositivo 6. Arquitetura do Exercício (2/2)

Arquitetura do Exercício (2/2)

- O cliente descobre o periférico, estabelece ligação, e lê ou escreve características para obter as leituras dos sensores.
- Esta divisão de papéis reproduz a arquitetura típica de uma solução de IoT: um nó restrito que recolhe dados e um nó mais capaz que os agrega e encaminha.

Porque esta divisão é representativa. O cliente descobre, liga e lê ou escreve características. A divisão reproduz a arquitetura típica de uma solução de IoT: um nó restrito que recolhe dados e um nó capaz que os agrega e encaminha. Reconhecer este padrão é útil, porque ele reaparece com outras tecnologias ao longo do semestre.

Diapositivo 7. Bibliotecas e Wrappers para BLE (1/2)

Bibliotecas e Wrappers para BLE (1/2)

- **BlueZ**: pilha protocolar Bluetooth oficial do Linux. Suporta todos os protocolos e camadas centrais do Bluetooth. Os pacotes `bluez-utils` e `bluez-firmware` fornecem os utilitários de baixo nível e o firmware necessário.
- **bluepy**: interface Python para BLE, construída sobre o BlueZ. É a biblioteca usada nesta unidade curricular.
- **PyGATT**: alternativa em Python, com interface centrada nas operações GATT.

As opções disponíveis. O BlueZ é a pilha protocolar oficial do Linux e suporta todos os protocolos centrais do Bluetooth. A bluepy é uma interface Python construída sobre ele, e é a biblioteca adotada nesta unidade curricular. A PyGATT é uma alternativa centrada nas operações GATT.

Diapositivo 8. Bibliotecas e Wrappers para BLE (2/2)

Bibliotecas e Wrappers para BLE (2/2)

- **SimpleBLE:** biblioteca com API simplificada, orientada a casos de utilização comuns.
- A escolha da biblioteca condiciona o nível de abstração disponível, mas todas assentam, em Linux, sobre a mesma pilha BlueZ.

O que as bibliotecas têm em comum. A SimpleBLE oferece uma interface simplificada. A observação a reter é a última: a escolha da biblioteca condiciona o nível de abstração, mas em Linux todas assentam sobre a mesma pilha BlueZ. Daqui decorre uma consequência prática útil: se algo não funciona ao nível da biblioteca, vale a pena verificar primeiro ao nível do BlueZ, com as ferramentas da Aula 04.

Diapositivo 9. bluepy: Modelo de Classes (1/2)

bluepy: Modelo de Classes (1/2)

- **Peripheral:** representa um dispositivo periférico ao qual o programa se liga. É o ponto de entrada para todas as operações sobre um dispositivo concreto.
- **Scanner:** procura dispositivos BLE que estejam a anunciar-se na vizinhança.
- **ScanEntry:** representa um dispositivo encontrado durante o varrimento, com o respetivo endereço e dados de anúncio.
- **DefaultDelegate:** classe base cujos métodos são invocados quando ocorrem eventos assíncronos, como a descoberta de um dispositivo ou a chegada de uma notificação.
- **UUID:** representa os identificadores de serviços e características.

As classes que interessam. Peripheral representa um dispositivo ao qual o programa se liga. Scanner procura dispositivos que se anunciem. ScanEntry representa um dispositivo encontrado. DefaultDelegate é a classe base cujos métodos são invocados quando ocorrem eventos assíncronos. UUID representa os identificadores.

Diapositivo 10. bluepy: Modelo de Classes (2/2)**bluepy: Modelo de Classes (2/2)**

- **Service, Characteristic e Descriptor:** correspondem diretamente à hierarquia GATT estudada na aula anterior.
- O mapeamento entre estas classes e os conceitos GATT é direto, o que facilita a passagem da teoria à implementação.

A correspondência com a Aula 04. Service, Characteristic e Descriptor correspondem diretamente à hierarquia GATT estudada na aula anterior. Esta correspondência direta é o motivo pelo qual a teoria da Aula 04 tem de estar consolidada antes desta. Quem domina a hierarquia de atributos reconhece imediatamente o modelo de classes.

Diapositivo 11. Implementação de um Varredor (1/2)

Implementação de um Varredor (1/2)

- O varrimento é o primeiro passo de qualquer aplicação cliente: sem descobrir o periférico, não há ligação possível.
- Cria-se uma subclasse de `DefaultDelegate` e redefine-se o método `handleDiscovery`, que recebe o dispositivo encontrado e dois indicadores: se é um dispositivo novo e se traz dados novos.
- Instancia-se um `Scanner` e associa-se-lhe o delegado através do método `withDelegate`.
- Invoca-se `scan` com a duração pretendida, em segundos, e obtém-se a lista de dispositivos encontrados.

Como se implementa um varredor. O varrimento é o primeiro passo de qualquer aplicação cliente: sem descobrir o periférico, não há ligação possível. O procedimento tem quatro passos. Criar uma subclasse de `DefaultDelegate` e redefinir `handleDiscovery`. Instanciar um `Scanner`. Associar-lhe o delegado com `withDelegate`. Invocar `scan` com a duração pretendida.

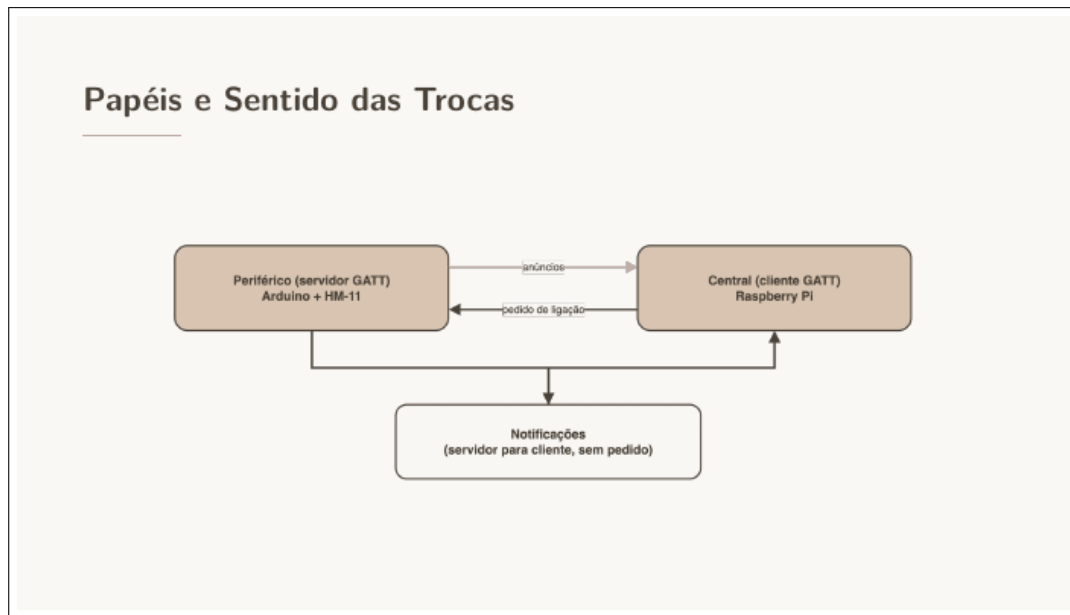
Diapositivo 12. Implementação de um Varredor (2/2)

Implementação de um Varredor (2/2)

- Para cada dispositivo, ficam disponíveis o endereço (`addr`), o tipo de endereço e a potência do sinal recebido (RSSI), em dBm.
- O valor de RSSI é útil para estimar proximidade, embora não constitua uma medida fiável de distância.

O que se obtém de cada dispositivo. Ficam disponíveis o endereço, o tipo de endereço e a potência do sinal recebido, em dBm. A advertência sobre o RSSI é importante: serve para estimar proximidade relativa, mas não constitui medida fiável de distância, porque a atenuação depende de obstáculos e reflexões.

Diapositivo 13. Papéis e Sentido das Trocas



Quem inicia cada tipo de troca. O diagrama mostra os dois sentidos. Os anúncios partem do periférico. O pedido de ligação parte do central. As notificações, uma vez estabelecida a ligação, partem novamente do periférico, sem que o cliente as peça de cada vez. É esta última seta que distingue as notificações de tudo o resto.

Diapositivo 14. Notificações: Conceito (1/2)

Notificações: Conceito (1/2)

- Ler periodicamente uma característica obriga o cliente a interrogar o servidor de forma repetida, mesmo quando não há dados novos, abordagem simples, mas ineficiente em consumo.
- As notificações invertem a iniciativa: é o servidor que informa o cliente sempre que o valor de uma característica se altera.
- No protocolo, distinguem-se notificações e indicações. A notificação não exige confirmação do cliente; a indicação exige, o que a torna mais fiável e mais dispendiosa.

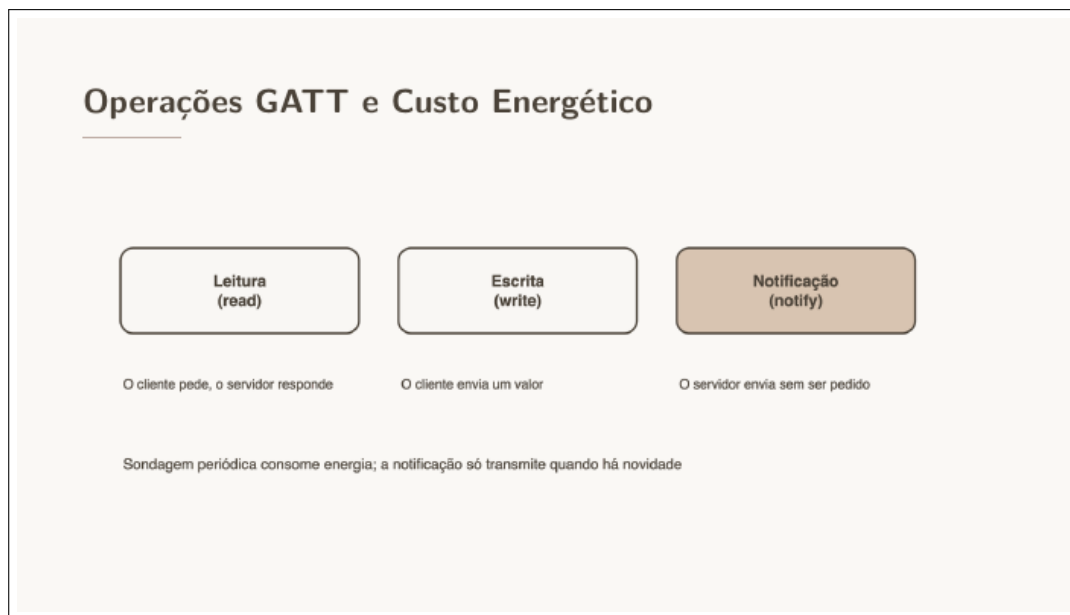
Porque a sondagem é ineficiente. Ler periodicamente uma característica obriga o cliente a interrogar o servidor de forma repetida, mesmo quando não há dados novos. As notificações invertem a iniciativa: é o servidor que informa quando o valor se altera. A distinção entre notificação e indicação deve ser retida: a primeira não exige confirmação, a segunda exige, o que a torna mais fiável e mais dispendiosa.

Diapositivo 15. Notificações: Conceito (2/2)**Notificações: Conceito (2/2)**

- Este mecanismo é particularmente adequado a sensores que reportam eventos esporádicos, como a deteção de movimento ou a ultrapassagem de um limiar.
- É também o mecanismo que sustenta a integração com um broker de mensagens, tratada mais adiante na unidade curricular.

Onde o mecanismo se aplica. As notificações são particularmente adequadas a sensores que reportam eventos esporádicos, como a deteção de movimento ou a ultrapassagem de um limiar. São também o mecanismo que sustenta a integração com um broker de mensagens, tratada na Aula 12. Vale a pena fixar esta ligação desde já.

Diapositivo 16. Operações GATT e Custo Energético



As três operações e o seu custo. O diagrama compara leitura, escrita e notificação. Nas duas primeiras o cliente toma a iniciativa. Na terceira, o servidor envia sem ser pedido. A observação em rodapé resume o essencial: a sondagem periódica consome energia mesmo quando nada mudou, ao passo que a notificação só transmite quando há novidade.

Diapositivo 17. Notificações: Implementação em bluepy (1/2)**Notificações: Implementação em bluepy (1/2)**

- Define-se uma subclasse de `DefaultDelegate` e redefine-se o método `handleNotification`, que recebe dois argumentos: o *handle* da característica que gerou a notificação e os dados recebidos.
- O *handle* permite distinguir a origem quando várias características estão configuradas para notificar.
- Cria-se o objeto `Peripheral` com o endereço do dispositivo e associa-se o delegado através de `setDelegate`.
- Obtém-se o serviço pretendido com `getServiceByUUID` e, a partir dele, a característica com `getCharacteristics`.

Os passos da implementação. Define-se uma subclasse de `DefaultDelegate`, redefinindo `handleNotification`, que recebe o identificador da característica e os dados. O identificador permite distinguir a origem quando várias características notificam. Cria-se o objeto `Peripheral`, associa-se o delegado, obtém-se o serviço com `getServiceByUUID` e a característica com `getCharacteristics`.

Diapositivo 18. Notificações: Implementação em bluepy (2/2)**Notificações: Implementação em bluepy (2/2)**

- Ativa-se a notificação escrevendo no descritor de configuração da característica.
- O programa entra depois num ciclo de espera, no qual chama `waitForNotifications`; sempre que chega uma notificação, o método `handleNotification` é invocado automaticamente.

Ativação e ciclo de espera. A notificação ativa-se escrevendo no descritor de configuração da característica. Este passo é frequentemente esquecido e é a causa mais comum de não se receber nada. O programa entra depois num ciclo que chama `waitForNotifications`. Sempre que chega uma notificação, `handleNotification` é invocado automaticamente.

Diapositivo 19. O Shield BLE HM-11 (1/2)**O Shield BLE HM-11 (1/2)**

- O exercício prático recorre a um shield de comunicações BLE instalado sobre o Arduino, baseado no módulo HM-11.
- A configuração do modo de funcionamento faz-se através da colocação de *jumpers* no shield, antes da montagem sobre a placa.
- O módulo comunica com o Arduino por porta série, o que exige a definição de uma porta série por software, uma vez que a porta série física está ocupada pela ligação ao computador.

O módulo HM-11. O exercício recorre a um shield instalado sobre o Arduino, baseado no módulo HM-11. A configuração do modo faz-se por jumpers, antes da montagem. O módulo comunica com o Arduino por porta série, o que obriga a definir uma porta série por software, uma vez que a física está ocupada pela ligação ao computador.

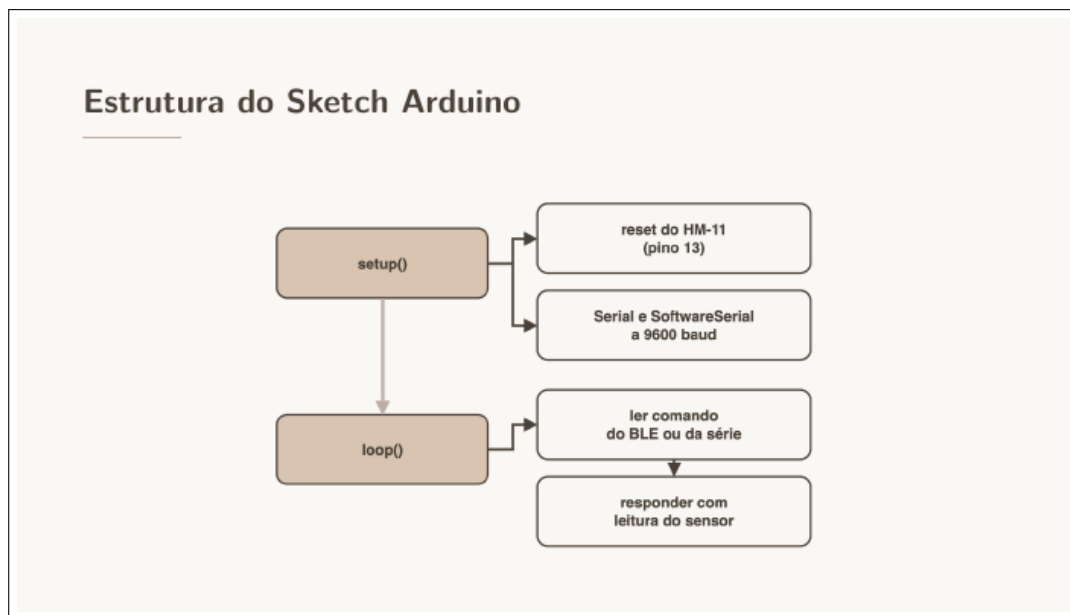
Diapositivo 20. O Shield BLE HM-11 (2/2)

O Shield BLE HM-11 (2/2)

- O pino de *reset* do módulo é controlado por uma saída digital do Arduino, o que permite reinicializar o módulo no arranque do programa.
- Ambas as portas série (a física, para o monitor série, e a virtual, para o módulo BLE) operam a 9600 bit/s.

Reset e débitos. O pino de reset do módulo é controlado por uma saída digital, o que permite reinicializar o módulo no arranque do programa. Ambas as portas série operam a 9600 bit/s. Divergências neste valor produzem texto ilegível, e são fonte frequente de erro no laboratório.

Diapositivo 21. Estrutura do Sketch Arduino



A estrutura do programa, em diagrama. O diagrama separa o que acontece uma vez, no setup, do que se repete no loop. No setup, reinicialização do módulo e abertura das duas portas. No loop, leitura de comando e resposta com a leitura do sensor.

Diapositivo 22. Estrutura do Programa Arduino (1/2)

Estrutura do Programa Arduino (1/2)

- Na função `setup`, o pino de `reset` é configurado como saída e o módulo é reiniciado através de um impulso, seguido da inicialização das duas portas série.
- Na função `loop`, o programa verifica se há dados disponíveis vindos do módulo BLE ou do monitor série, e lê o carácter recebido.
- O carácter recebido funciona como comando: `t` para temperatura, `h` para humidade, `l` para luminosidade e `X` para o conjunto de todas as grandezas.

O que faz o setup e o que faz o loop. No `setup` configura-se o pino de `reset`, reinicializa-se o módulo por impulso e abrem-se as duas portas série. No `loop` verifica-se se há dados do módulo BLE ou do monitor série. O carácter recebido funciona como comando: `t` para temperatura, `h` para humidade, `l` para luminosidade e `X` para o conjunto.

Diapositivo 23. Estrutura do Programa Arduino (2/2)

Estrutura do Programa Arduino (2/2)

- A cada comando corresponde uma resposta enviada simultaneamente para o módulo BLE e para o monitor série, o que facilita o diagnóstico durante o desenvolvimento.
- Cada comando define ainda um fator de escala aplicado ao atraso do ciclo, o que permite ajustar a periodicidade de resposta consoante a grandeza pedida.

Uma decisão de projeto que facilita o diagnóstico. A resposta é enviada simultaneamente para o módulo BLE e para o monitor série. Esta duplicação é deliberada e permite verificar o funcionamento do sketch sem envolver a comunicação BLE. Cada comando define ainda um fator de escala aplicado ao atraso do ciclo.

Diapositivo 24. Verificação do Lado do Gateway (1/2)

Verificação do Lado do Gateway (1/2)

- O comando `hcitool dev` lista os adaptadores Bluetooth disponíveis no sistema, com os respectivos identificadores e endereços físicos.
- O comando `hcitool lescan` inicia o varrimento de dispositivos BLE na vizinhança, apresentando os endereços encontrados e, quando disponível, o nome anunciado pelo dispositivo.
- Identificado o endereço do periférico, estabelece-se a ligação com `gatttool -b <endereço> -I`, que abre a sessão em modo interativo.
- Dentro dessa sessão, o comando `connect` estabelece a ligação efetiva ao dispositivo.

Verificar antes de programar. O comando `hcitool dev` lista os adaptadores. O `hcitool lescan` procura dispositivos. Identificado o endereço, estabelece-se a ligação com `gatttool` em modo interativo, e dentro dessa sessão o comando `connect` liga efetivamente.

Diapositivo 25. Verificação do Lado do Gateway (2/2)

Verificação do Lado do Gateway (2/2)

- Segue-se a descoberta dos serviços e características expostos, e a leitura ou escrita dos respetivos valores.
- Esta verificação por linha de comandos deve preceder o desenvolvimento em Python: confirma que a ligação é possível antes de se introduzir a complexidade do código.

A razão da ordem. Segue-se a descoberta dos serviços e características, e a leitura ou escrita dos valores. A justificação apresentada no diapositivo é o ponto a reter: esta verificação deve preceder o desenvolvimento em Python, porque confirma que a ligação é possível antes de se introduzir a complexidade do código.

Diapositivo 26. Método de Trabalho Recomendado (1/2)

Método de Trabalho Recomendado (1/2)

- Preparar e testar o hardware isoladamente: verificar o funcionamento do sketch através do monitor série, antes de envolver a comunicação BLE.
- Confirmar a visibilidade do periférico com `hcitool lescan`, garantindo que o módulo está efetivamente a anunciar-se.
- Estabelecer a ligação manualmente com `gatttool` e explorar os serviços e características disponíveis.

Os três primeiros passos. Testar o hardware isoladamente pelo monitor série. Confirmar a visibilidade do periférico com `hcitool lescan`. Estabelecer a ligação manualmente com `gatttool` e explorar os serviços.

Diapositivo 27. Método de Trabalho Recomendado (2/2)

Método de Trabalho Recomendado (2/2)

- Só depois passar ao desenvolvimento em Python, começando pelo varrimento e progredindo para a leitura de características e para as notificações.
- Esta progressão isola a origem dos problemas: uma falha detetada no passo três não pode ser atribuída ao código Python, que ainda não foi escrito.

Porque a progressão importa. Só depois se passa ao desenvolvimento em Python, do varrimento para a leitura e para as notificações. A justificação é metodológica e generalizável: esta progressão isola a origem dos problemas. Uma falha detetada no terceiro passo não pode ser atribuída ao código Python, que ainda não existe.

Diapositivo 28. Síntese da Aula (1/2)

Síntese da Aula (1/2)

- A programação BLE traduz diretamente a hierarquia GATT: serviços, características e descritores têm correspondência nas classes das bibliotecas usadas.
- Em Linux, a pilha BlueZ é a base sobre a qual assentam as bibliotecas de mais alto nível, entre as quais a bluepy usada nesta unidade curricular.
- O padrão de delegação (subclasse de DefaultDelegate com redefinição de `handleDiscovery` ou `handleNotification`) é o mecanismo através do qual a aplicação reage a eventos assíncronos.

Síntese, primeira parte. A programação BLE traduz diretamente a hierarquia GATT. Em Linux, o BlueZ é a base sobre a qual assentam as bibliotecas de mais alto nível. O padrão de delegação, com subclasse de DefaultDelegate e redefinição de `handleDiscovery` ou `handleNotification`, é o mecanismo através do qual a aplicação reage a eventos assíncronos.

Diapositivo 29. Síntese da Aula (2/2)**Síntese da Aula (2/2)**

- As notificações substituem a interrogação periódica, reduzindo o consumo e a latência, e são o mecanismo natural para encaminhar leituras para um middleware de mensagens.
- Nas aulas seguintes, o papel de cliente passa para um Raspberry Pi, que atuará como gateway entre a rede BLE e a rede IP.

Síntese, segunda parte. As notificações substituem a interrogação periódica, reduzindo consumo e latência, e são o mecanismo natural para encaminhar leituras para um middleware de mensagens. Nas aulas seguintes o papel de cliente passa para um Raspberry Pi, que atuará como gateway entre a rede BLE e a rede IP.

Diapositivo 30. Fontes

Fontes

Gonçalves, P. *Apresent5 — BLE Dev e guia 4 — arduinoBLE*, TCIoT, edições anteriores. Base do modelo de classes bluepy, do varredor, das notificações e do exercício com o shield HM-11.

Bluetooth SIG. *Bluetooth Core Specification*. Definição das operações GATT de leitura, escrita, notificação e indicação.

Documentação da biblioteca bluepy: ianharvey.github.io/bluepy-doc/

Origem do material. A base é o material da unidade curricular de edições anteriores, complementado pela especificação do Bluetooth SIG quanto às operações GATT e pela documentação da bluepy.

Verificação Final

Antes de dar a matéria por estudada, verifique se consegue responder ao seguinte sem consultar os diapositivos.

1. Explicar a correspondência entre as classes da bluepy e a hierarquia GATT da Aula 04.
2. Descrever os quatro passos necessários para implementar um varredor.
3. Distinguir notificação de indicação e indicar qual delas exige confirmação do cliente.
4. Indicar o passo de ativação sem o qual nenhuma notificação é recebida.
5. Justificar a necessidade de uma porta série por software no Arduino.
6. Explicar por que motivo a verificação com gatttool deve preceder o desenvolvimento em Python.
7. Indicar o que se pode e o que não se pode concluir a partir de um valor de RSSI.

Referências

- Gonçalves, P. *Apresent5 — BLE Dev e guia 4 — arduinoBLE*, TCIoT, edições anteriores.
- Bluetooth SIG. *Bluetooth Core Specification*.
- Documentação da biblioteca bluepy: ianharvey.github.io/bluepy-doc/